



Short-Term Internship Project Report

On the title

"Display of Incremental Rotary Encoder Output on GUI Using PIC 16-bit Microcontroller"

submitted in accordance with the requirements of
the Visiting Student Programme at ARIES (VSPA)

Internship period
05.01.2026 – 30.01.2026

Suraj Khanduri
(intern)
Department of Electronics and
Communication Engineering,
Govind Ballabh Pant Institute of
Engineering and Technology,
Pauri.

Mr. Shobhit Yadava
(Guide)
Engineer-E,
Aryabhata Research Institute of
Observational Sciences (ARIES),
Nainital.

DECLARATION

I hereby declare that the project report entitled

**“Display of Incremental Rotary Encoder Output on GUI Using
PIC 16-bit Microcontroller”**

is an original record of the work carried out by me during my short-term internship at the Aryabhata Research Institute of Observational Sciences (ARIES), Nainital, under the Visiting Student Programme at ARIES (VSPA), during the period 05.01.2026 to 30.01.2026, under the guidance of Mr. Shobhit Yadava.

All sources of information and data used in this report have been acknowledged.

I understand that any violation of academic integrity may result in appropriate disciplinary action as per the rules and regulations of ARIES.

Place: Manora Peak, Nainital.

Date: 30.01.2026



Suraj Khanduri

(intern)

Department of Electronics and Communication
Engineering,
Govind Ballabh Pant Institute of Engineering and
Technology, Pauri

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Mr. Shobhit Yadava, Aryabhata Research Institute of Observational Sciences (ARIES), for giving me the opportunity to carry out this short-term project under the Visiting Student Programme at ARIES (VSPA).

I am deeply thankful for his constant guidance, encouragement, and insightful technical discussions, which were crucial in shaping my understanding throughout the course of this work.

I sincerely thank the faculty members of the Department of Electronics and Communication Engineering, Govind Ballabh Pant Institute of Engineering and Technology, Pauri, for their academic support and encouragement throughout my undergraduate studies.

Finally, I express my heartfelt gratitude to my family and friends for their constant support, motivation, and understanding during this period.

ABSTRACT

This report presents the work carried out during a short-term internship at the Aryabhatta Research Institute of Observational Sciences (ARIES), Manora Peak, Nainital, under the Visiting Student Programme at ARIES (VSPA).

This project focuses on the acquisition and visualization of angular position and direction data from an incremental rotary encoder using a PIC 16-bit microcontroller. An industrial-grade, incremental optical rotary encoder is interfaced with a PIC microcontroller through the Quadrature Encoder Interface (QEI) to accurately decode encoder signals. The decoded position and direction data are processed and transmitted using serial communication protocols.

To improve modularity and reliability, a dual-microcontroller architecture is adopted. The first microcontroller is dedicated to encoder signal processing, while the second microcontroller handles data communication. Encoder data is transferred between the two microcontrollers using the Serial Peripheral Interface (SPI) protocol, and the processed output is transmitted to a PC-based graphical user interface (GUI) through UART communication for real-time display.

The project also studies the relevance of encoder feedback in practical motion systems such as stepper motor-driven linear actuators using ball screw and linear guide mechanisms. This establishes a foundation for future expansion toward linear position monitoring and closed-loop motion control. The current stage of the project includes theoretical study, hardware setup, and interface planning, with firmware implementation and GUI integration in progress.

TABLE OF CONTENT

1. Introduction	1
2. Optical Encoder	2
2.1 Construction	2
2.2 Working Principle	3
2.3 Advantages of Optical Encoders.....	3
2.4 Limitations.....	3
3. Incremental Rotary Encoder (Project Specific)	4
3.1 Signal Structure	4
3.2 Direction Sensing.....	4
3.3 Position Measurement	5
3.4 Suitability for This Project	5
4. PIC 16-bit Microcontroller	6
4.1 Architecture Overview	6
4.2 On-Chip Peripherals	7
4.3 Role in the Project.....	7
4.4 Why PIC 16-bit MCU is Chosen.....	7
5. Serial Communication Protocol	8
5.1 UART Communication Basics.....	8
5.2 Data Transmission Format	8
6. Quadrature Encoder Interface (QEI).....	9
6.1 QEI Operation.....	9
6.2 Counting Modes	10
6.3 Benefits of Using QEI	10
6.4 Application in This Project.....	10
7. Hall Effect Vane Sensor.....	11
7.1 Working Principle	11
7.2 Comparison with Optical Encoder	11
7.3 Relevance to the Project	12
8. Stepper Motor and Actuator Assembly	13
8.1 Construction and Working	13
8.3 Importance in Encoder-Based Systems	14
9. Prototype (Stepper Outputs)	15
9.1 Unipolar Stepper Motor	15
Program Snippets.....	16
9.2 Bipolar Stepper Motor	17
Program Snippets.....	18
10. Rotary Encoder GUI Using Python (Tkinter + PySerial).....	20
10.1 Program Objective.....	20
10.2 Import of Required Libraries	20
10.3 COM Port Detection	20
10.4 Serial Port Configuration	20
10.5 GUI Window Initialization	21

10.6 Encoder Data Display Element	21
10.7 Serial Data Acquisition	21
10.8 Data Processing and Validation.....	21
10.9 Real-Time GUI Update and Execution	22
11.Conclusion	23

1. Introduction



Figure 1

Overall system concept of incremental rotary encoder-based GUI using PIC 16-bit MCU. Rotary encoders are widely used for position and motion sensing devices in embedded and industrial systems. This project focuses on designing and implementing a system to read an **incremental rotary encoder** using a **PIC 16-bit microcontroller** and displaying the encoder output on a **Graphical User Interface (GUI)** through serial communication. The project integrates concepts of sensing, embedded processing, and human– machine interfacing.

The work involves understanding encoder principles, configuring the PIC MCU peripherals (especially QEI and UART), and transmitting processed data to a PC-based GUI for visualization.

2. Optical Encoder

An optical encoder is an electromechanical sensing device used to convert mechanical motion into electrical signals using light as the sensing medium. Optical encoders are widely used in precision motion control, robotics, automation, and instrumentation systems due to their high resolution and accuracy.

2.1 Construction

An optical encoder typically consists of the following key components, each playing a crucial role in converting mechanical motion into precise electrical signals.

Light Source:

An infrared LED is commonly used as the light source because of its stability, low power consumption, and long operating life. It emits a continuous and focused beam of light toward the sensing region of the encoder.

Code Disk:

The code disk is mounted directly on the rotating shaft and contains evenly spaced transparent and opaque segments. As the shaft rotates, these segments alternately allow and block the light beam. The number of segments on the disk determines the resolution of the encoder and directly affects measurement accuracy.

Photodetectors:

Photodetectors such as photodiodes or phototransistors are placed opposite the LED to detect the transmitted or blocked light. The varying light intensity caused by the rotating disk is converted into corresponding electrical signals. In incremental encoders, multiple photodetectors are arranged with a phase difference to generate quadrature signals for direction detection.

Signal Conditioning Circuitry:

The raw electrical signals from the photodetectors are typically weak and susceptible to noise. Signal conditioning circuitry, including amplifiers, filters, comparators, and Schmitt triggers, is used to clean and shape these signals into sharp, noise-free digital pulses. These conditioned outputs can be directly interfaced with a microcontroller for accurate measurement of speed, position, and direction.

2.2 Working Principle

As the encoder shaft rotates, the code disk rotates along with it. The transparent segments allow light to pass through and reach the photodetector, while opaque segments block the light. This results in a sequence of ON–OFF signals corresponding to shaft rotation. These signals are converted into square-wave digital pulses.

The number of pulses generated per revolution determines the resolution of the encoder. Higher resolution encoders provide more precise position feedback.

2.3 Advantages of Optical Encoders

- High accuracy and resolution
- Non-contact sensing (minimal wear)
- Good repeatability
- Suitable for high-speed operation

2.4 Limitations

- Sensitive to dust, oil, and environmental contamination
- Requires precise alignment
- Higher cost compared to magnetic encoders

3. Incremental Rotary Encoder (Project Specific)



Figure 2

Quadrature signal generation of incremental rotary encoder (Channel A and Channel B). An incremental rotary encoder is a type of optical encoder that generates pulses relative to shaft movement rather than providing an absolute position.

3.1 Signal Structure

Incremental encoders typically generate: - **Channel A**: Square-wave pulse train - **Channel B**: Square-wave pulse train phase-shifted by 90° with respect to Channel A - **Index Channel (Z)** (optional): Generates one pulse per revolution for reference.

3.2 Direction Sensing

The phase difference between Channel A and Channel B is used to determine the direction of rotation: - Channel A leading Channel B $\rightarrow$ Clockwise rotation - Channel B leading Channel A $\rightarrow$ Counter-clockwise rotation

This method is known as **quadrature encoding**.

3.3 Position Measurement

Position is determined by counting the pulses generated during rotation. The count value increases or decreases based on the direction of rotation. The angular position can be calculated using:

$$\text{Angular Position} = (\text{Pulse Count} / \text{Pulses per Revolution}) \times 360^\circ$$

3.4 Suitability for This Project

Incremental rotary encoders are chosen for this project due to: - Simple interfacing with PIC QEI module - Low cost and easy availability - Adequate resolution for GUI-based monitoring - Real-time direction and position feedback. An incremental rotary encoder is well suited for this project because it generates pulse signals that directly represent shaft rotation, making it ideal for real-time measurement of speed, position, and direction. Its quadrature outputs (A and B) allow the PIC 16-bit microcontroller to accurately determine clockwise and counterclockwise motion using interrupts or timers, matching the project's encoder processing logic. The high pulse resolution enables precise position tracking and smooth speed calculation, while the simple digital interface ensures reliable integration with the PIC and easy transmission of motion data to the Python-based GUI. This makes the incremental rotary encoder a practical and industry-relevant feedback device for validating motion control and encoder simulation in the project.

4. PIC 16-bit Microcontroller

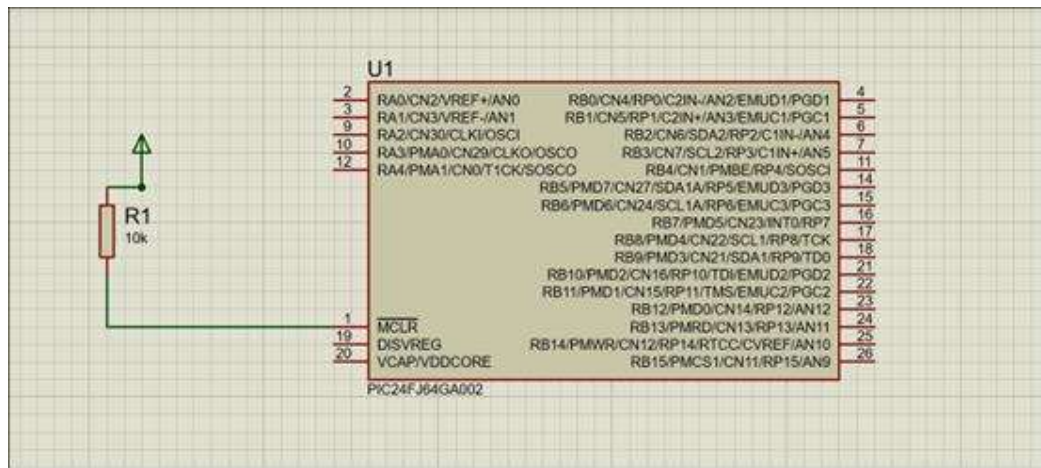


Figure 3

Role of PIC 16-bit MCU in encoder signal processing and communication. PIC 16-bit microcontrollers, such as the PIC24 and dsPIC series, are designed for real-time embedded control applications requiring efficient processing and rich peripheral support.

4.1 Architecture Overview

The PIC 16-bit microcontroller is based on a **modified Harvard architecture**, where program memory and data memory are separated, allowing simultaneous instruction to fetch and data access for higher execution speed. It uses a **16-bit CPU core** with a wide data path, enabling faster arithmetic operations and efficient handling of timers, counters, and control algorithms. The architecture includes **Flash program memory**, **RAM**, and **EEPROM** for flexible code storage and data retention. A rich set of **peripherals** is integrated on-chip, such as multiple 16-bit timers, external interrupts, capture/compare/PWM modules, UART, SPI, and I²C interfaces, which support real-time motion sensing and communication tasks. The MCU also features a **configurable oscillator system**, power-saving modes, and a vectored interrupt controller that ensures fast and predictable interrupt response. This architecture makes PIC 16-bit controllers well suited for deterministic, real-time embedded applications like encoder processing and motor control.

- 16-bit CPU core
- Harvard Architecture
- High instruction throughput
- Separate program and data memory space

4.2 On-Chip Peripherals

PIC 16-bit MCUs include: - UART for serial communication - SPI and I2C interfaces - Timers and counters - Quadrature Encoder Interface (QEI) - Interrupt controller

4.3 Role in the Project

In this project, the PIC 16-bit MCU performs the following functions: - Interfaces directly with the incremental encoder using QEI pins - Decodes quadrature signals in hardware - Maintains position count and direction - Transmits processed data to a PC via UART.

4.4 Why PIC 16-bit MCU is Chosen

The PIC 16-bit microcontroller is chosen for this project because it provides the performance, precision, and peripheral support required for real-time motion sensing and control. Compared to 8-bit controllers, the 16-bit architecture offers higher instruction throughput and better timer resolution, which is critical for accurate counting of high-frequency encoder or Hall-effect sensor pulses and precise speed calculation. It includes multiple 16-bit timers, external interrupt modules, and capture/compare features that simplify direction detection and position measurement. The built-in hardware UART ensures reliable, low-latency communication with the Python-based GUI, enabling real-time data visualization. Additionally, 16-bit PIC MCUs offer sufficient Flash and RAM for structured firmware, stable operation in continuous running systems, low power consumption, and strong toolchain support through MPLAB X and PICKit, making them a robust and industry-relevant choice for embedded motion-control applications.

- Hardware QEI support
- Reliable UART communication
- Suitable for real-time signal processing
- Widely used in academic and industrial applications

5. Serial Communication Protocol

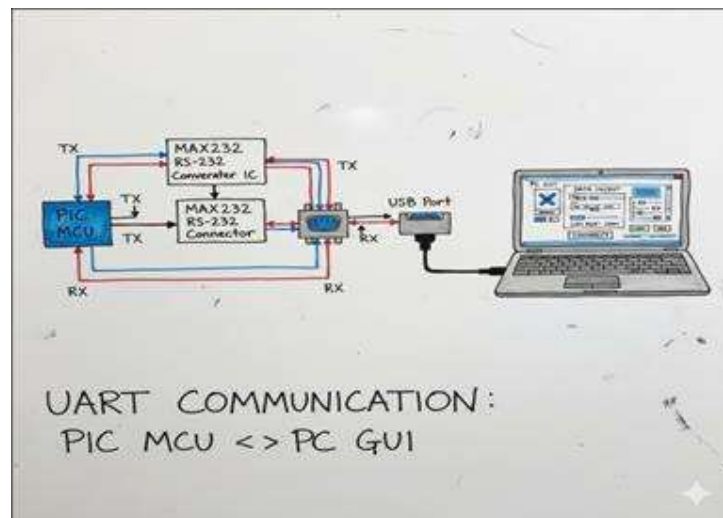


Figure 4

UART-based communication between PIC MCU and PC GUI. Serial communication is used to transfer encoder data from the microcontroller to a computer-based GUI.

5.1 UART Communication Basics

UART (Universal Asynchronous Receiver/Transmitter) is a widely used serial communication protocol that enables data exchange between a microcontroller and external devices without a shared clock. Communication is asynchronous, meaning both devices agree on parameters such as baud rate, data bits, parity, and stop bits before transmission begins. Data is sent in the form of frames that typically include a **start bit** (indicating the beginning of data transmission), **data bits** (usually 8 bits representing the actual information), an optional **parity bit** for error checking, and one or more **stop bits** to signal the end of the frame. UART uses two main lines: **TX (transmit)** to send data and **RX (receive)** to receive data, with a common ground reference. Its simplicity, low hardware requirement, and reliable point-to-point communication make UART ideal for interfacing microcontrollers with PCs, sensors, and GUI-based applications, such as real-time data transfer between a PIC MCU and a Python interface.

5.2 Data Transmission Format

UART data is sent in frames consisting of a **start bit**, followed by **data bits** (usually 8 bits, LSB first), an optional **parity bit** for error checking, and one or more **stop bits** to indicate the end of transmission. This format enables reliable asynchronous communication between devices without a shared clock.

6. Quadrature Encoder Interface (QEI)

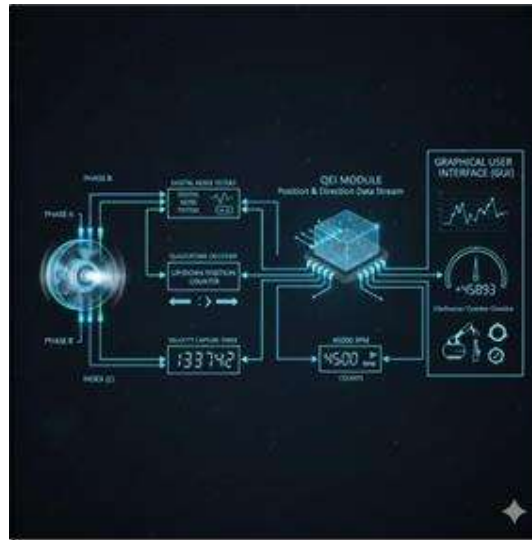


Figure 5

QEI block representation for decoding quadrature signals. The Quadrature Encoder Interface (QEI) is a dedicated hardware module in PIC 16-bit microcontrollers used for decoding incremental encoder signals.

6.1 QEI Operation

Quadrature Encoder Interface (QEI) operates by decoding two square-wave signals (Channel A and Channel B) generated by an incremental encoder, which are phase-shifted by 90° . By monitoring the sequence and relative timing of these signals, the QEI determines the direction of rotation (clockwise or counterclockwise) and counts pulses to calculate position and speed. The hardware QEI module automatically handles pulse counting, direction detection, and index pulse processing, reducing CPU load, and enabling accurate, real-time motion tracking in motor control applications.

6.2 Counting Modes

- **x1 Mode:** Counts on one edge of Channel A
- **x2 Mode:** Counts on both edges of Channel A
- **x4 Mode:** Counts on all edges of Channels A and B

6.3 Benefits of Using QEI

- Eliminates software-based decoding
- Improves accuracy and speed
- Reduces CPU overhead
- Reliable at high rotation speeds

6.4 Application in This Project

The QEI module is configured to continuously track encoder position and direction, which is then transmitted to the GUI.

7. Hall Effect Vane Sensor

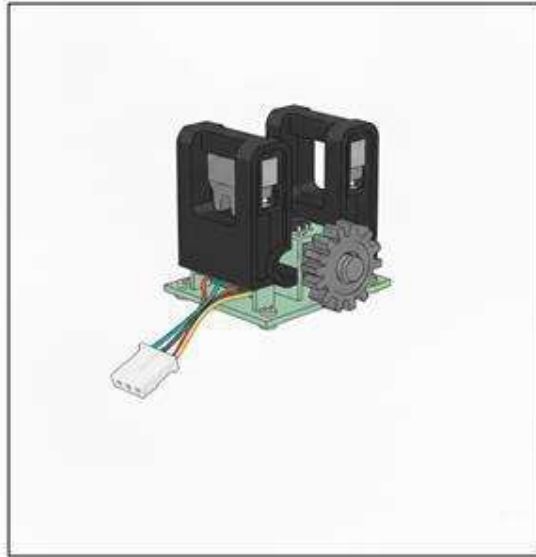


Figure 6

Basic working principle of Hall effect vane sensor. A Hall effect vane sensor is a magnetic sensing device that detects changes in magnetic field caused by the movement of a ferromagnetic vane.

7.1 Working Principle

When a magnetic field passes through a Hall sensor, a voltage proportional to the field's strength is generated. The presence or absence of a vane modulates this field, producing digital output pulses.

7.2 Comparison with Optical Encoder

- Less sensitive to dust and contaminants
- Lower resolution compared to optical encoders
- Suitable for harsh environments

7.3 Relevance to the Project

Hall effect sensors are studied as an alternative feedback mechanism for motion detection in industrial environments where optical encoders may fail. A Hall-effect vane sensor is relevant to this project because it provides a non-contact, reliable method of generating encoder-like digital pulses that can be directly interfaced with the PIC 16-bit microcontroller for real-time speed, position, and direction feedback. When coupled with a rotating vane or motor shaft, the sensor output can be used to validate rotary encoder logic, measure motor or linear stage motion, and transmit accurate motion data to the Python GUI. This enhances the project by moving beyond pure signal simulation to hardware-verified feedback, making the system more robust and suitable for practical motion-control and automation applications.

8. Stepper Motor and Actuator Assembly

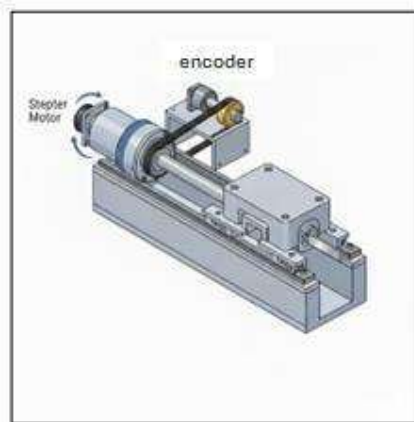


Figure 7

Stepper motor and encoder-based positioning concept. A stepper motor is an electromechanical actuator that converts electrical pulses into discrete mechanical steps.

$$\text{Linear displacement (mm)} = (C / N) \times L$$

- Ball screw lead = **L mm/revolution**
- Encoder resolution = **N pulses/rev**
- Encoder count = **C pulses**

8.1 Construction and Working

A stepper motor consists of: - Stator with multiple windings - Rotor with permanent magnets or soft iron core

Each electrical pulse causes the motor to move one step. The step angle depends on the motor design.

8.2 Characteristics

- High positional accuracy
- Open-loop control capability
- Good low-speed torque

8.3 Importance in Encoder-Based Systems

In practical systems, stepper motors are often combined with encoders to: - Verify commanded position - Detect missed steps - Implement closed-loop control

Understanding stepper motors helps establish the practical relevance of encoder feedback.

9. Prototype (Stepper Outputs)

9.1 Unipolar Stepper Motor

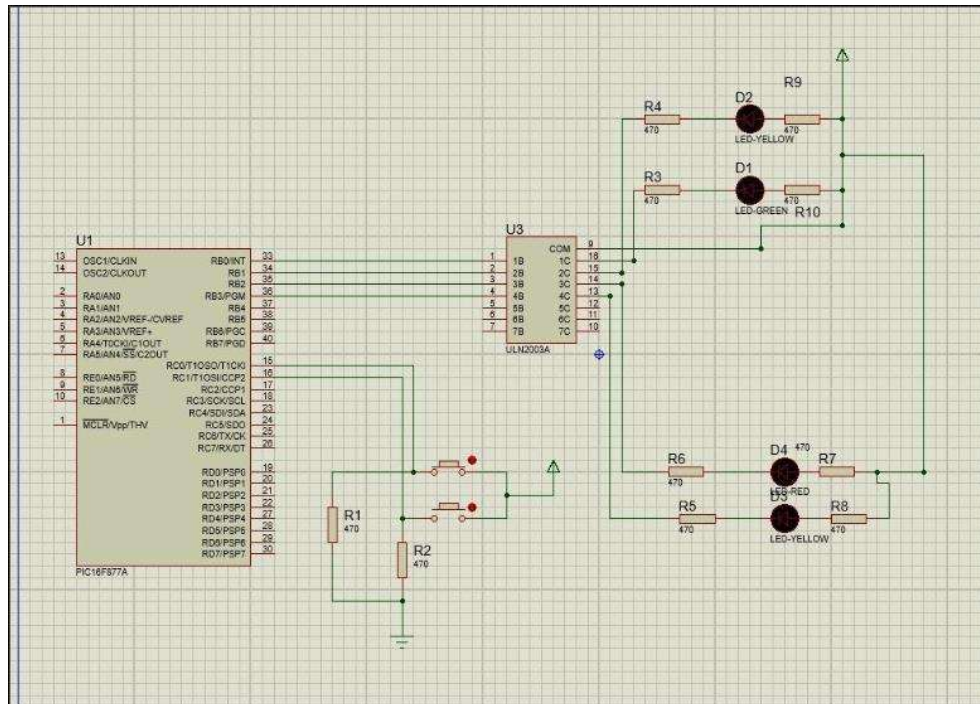


Figure 8

This circuit acts as a **display/indicator for stepper motor driving inputs** generated by the PIC16F877A. The microcontroller outputs step sequences on PORTB, which are fed into the ULN2003A driver. The ULN2003A provides current amplification and isolation, allowing the logic-level step signals to safely drive the LED indicators. Each LED (red, yellow, and green) with its 470 Ω current-limiting resistor represents a specific stepper motor phase or input line, visually showing the stepping sequence (CW or CCW) in real time. The push buttons connected to the PIC inputs allow users to control the stepping direction or operation. This setup is useful for debugging and understanding stepper motor excitation patterns before connecting an actual motor.

Clockwise (CW) Rotation

Step	PORTB (Hex)	RB3	RB2	RB1	RB0	LED ON
1	0x01	0	0	0	1	LED1
2	0x02	0	0	1	0	LED2
3	0x04	0	1	0	0	LED3
4	0x08	1	0	0	0	LED4

Counter-Clockwise (CCW) Rotation

Step	PORTB (Hex)	RB3	RB2	RB1	RB0	LED ON
1	0x08	1	0	0	0	LED4
2	0x04	0	1	0	0	LED3
3	0x02	0	0	1	0	LED2
4	0x01	0	0	0	1	LED1

Program Snippets

<pre> 1 // PIC16F877A Configuration Bit Settings 2 // 1. OSCILLATOR Configuration Bit Settings 3 // 2. WATCHDOG Configuration Bit Settings 4 // 3. VDD Configuration Bit Settings 5 6 // CONFIG 7 #pragma config FOSC = HS // Oscillator Selection 8 // 1. OSCILLATOR Selection 9 #pragma config WDT = OFF // Watchdog Timer 10 // 1. WDT Selection 11 #pragma config PWRTE = ON // Power-up Timer 12 // 1. PWRTE Selection 13 #pragma config BOR = ON // Brown-out Reset 14 // 1. BOR Selection 15 // 1. BOR Selection 16 // 1. BOR Selection 17 // 1. BOR Selection 18 // 1. BOR Selection 19 // 1. BOR Selection 20 // 1. BOR Selection 21 // 1. BOR Selection 22 // 1. BOR Selection 23 // 1. BOR Selection 24 // 1. BOR Selection 25 // 1. BOR Selection 26 // 1. BOR Selection 27 // 1. BOR Selection </pre>	<pre> 28 TRISB = 0x00; 29 // Set RB0, RB1, RB2, RB3 to be output (initially) 30 RB0 = 0x00; 31 // Set RB0, RB1, RB2, RB3 to be output pins 32 TRISB = 0x00; 33 34 // Create the system's main routine 35 while(1) 36 { 37 // While button is pressed, rotate CW 38 while(RC0) 39 { 40 // Rotate clockwise 41 char step_seq[4] = {0x01, 0x02, 0x04, 0x08}; 42 43 // Rotate clockwise 44 for(i=0; i<4; i++) 45 { 46 // Delay 100ms 47 delay_ms(100); 48 } 49 // Rotate clockwise 50 while(RC1) 51 { 52 // Rotate clockwise 53 for(i=0; i<4; i++) 54 { 55 // Delay 100ms 56 delay_ms(100); 57 } 58 } 59 } 60 } </pre>
--	--

Figure 9

This program configures the **PIC16F877A** to generate stepper-motor drive patterns and display them using LEDs. PORTB is set as an output and sends a **4-step excitation sequence** (0x01, 0x02, 0x04, 0x08) that represents individual stepper phases; these signals go through the ULN2003A and light the corresponding LEDs, visually indicating the active coil. RC0 and RC1 are configured as inputs and act as direction control buttons: when **RC0 is pressed**, the controller outputs the sequence in forward order (clockwise rotation), and when **RC1 is**

pressed, it outputs the reverse pattern using right-shift logic (counterclockwise rotation). A 100 ms delay between steps controls the stepping speed. Overall, the code demonstrates basic stepper motor control while using LEDs as a real-time display of the driving inputs.

9.2 Bipolar Stepper Motor

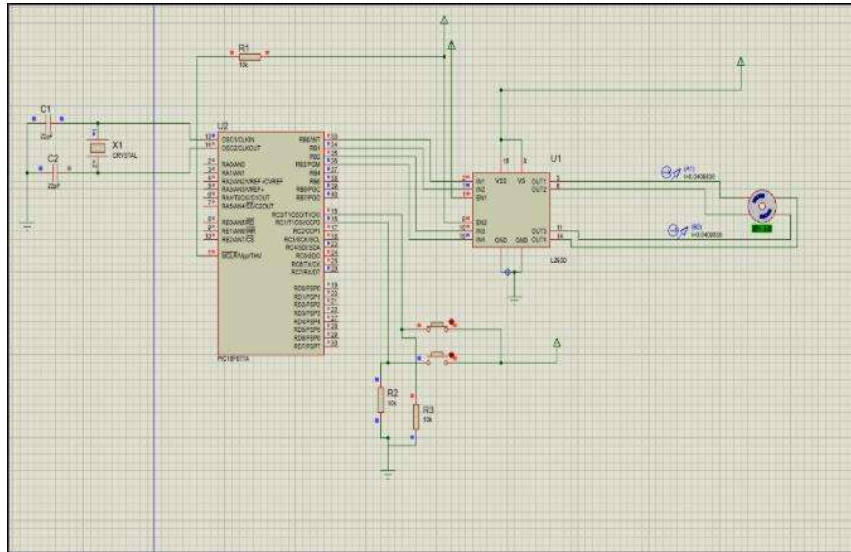


Figure 10

This circuit shows the **actual stepper motor drive implementation** using a **PIC16F877A** and an **L293D dual H-bridge driver**. The PIC is clocked using an external crystal with load capacitors, ensuring stable operation, and its PORTB pins generate the stepper motor excitation sequence. These logic signals are fed to the IN1–IN4 pins of the L293D, while the enabled pins are held active, allowing the driver to amplify the signals and supply sufficient current to the stepper motor windings. The L293D isolates and protects the microcontroller from motor back-EMF while providing bidirectional current control for clockwise and counterclockwise rotation. Push buttons connected to the PIC inputs with pull-down resistors allow the user to control the motor direction, corresponding directly to the stepper sequence logic used in the program. Overall, this circuit converts the previously displayed step patterns into real motor motion, demonstrating complete stepper motor control using a microcontroller and driver IC.

CW sequence (RC0 pressed)

Step	PORTB (Binary)	RB3	RB2	RB1	RB0
1	00000101	0	1	0	1
2	00000110	0	1	1	0
3	00001010	1	0	1	0
4	00001001	1	0	0	1

CCW sequence (RC1 pressed)

Step	PORTB (Binary)	RB3	RB2	RB1	RB0
1	00001001	1	0	0	1
2	00001010	1	0	1	0
3	00000110	0	1	1	0
4	00000101	0	1	0	1

Program Snippets

This program is developed for a **PIC microcontroller** to control the rotation of a **stepper motor** using output signals from **PORTB** and direction control inputs applied to **RC0** and **RC1**. The configuration bits are set to operate the microcontroller with a **10 MHz high-speed crystal oscillator**, while the **watchdog timer is disabled** and **low-voltage programming is turned off** to ensure stable operation.

PORTB is configured as an output port to drive the stepper motor coils, whereas **PORTC** is configured as an input port to read the direction control signals. The program executes within an infinite while(1) loop, allowing continuous monitoring of the input pins and uninterrupted motor operation.

When **RC0 is HIGH**, the microcontroller outputs a predefined sequence of four-bit patterns on PORTB

(00000101, 00000110, 00001010, 00001001). This stepping sequence energizes the motor coils in the correct order to produce **clockwise rotation**. Conversely, when **RC1 is HIGH**, the stepping sequence is applied in reverse order, resulting in **counterclockwise rotation** of the motor.

A fixed delay is introduced between successive steps to control the **speed of the stepper motor**. The continuous execution of the loop ensures smooth and sustained rotation of the motor as long as the corresponding control input remains active.


```

1 #include <x86.h>
2
3 // Configuration Bits (Standard for HS Crystal)
4 #pragma config FOSC = HS
5 #pragma config WDTE = OFF
6 #pragma config FWRTE = OFF
7 #pragma config BOREN = ON
8 #pragma config LVP = OFF
9
10 #define _XTAL_FREQ 20000000 // 20MHz Crystal
11
12 void main() {
13     TRISB = 0x00; // Set PORTB as Output
14     PORTB = 0x00; // Initialize all pins LOW
15
16     TRISC = 1;
17
18
19     while(1) {
20         while(RC0)
21         {
22             // Step 1
23             PORTB = 0b00000101; // RB2 and RB0 HIGH
24             delay_ms(100);
25             // Step 2
26             PORTB = 0b00000110; // RB2 and RB1 HIGH
27             delay_ms(100);
28             // Step 3
29             PORTB = 0b00001010; // RB3 and RB1 HIGH
30             delay_ms(100);
31             // Step 4
32             PORTB = 0b00001001; // RB3 and RB0 HIGH
33             delay_ms(100);
34         }
35         while(RC1)
36         {
37             //Step 1

```

Page 1

```

38         PORTB = 0b00001001; // RB3 and RB0 HIGH
39         delay_ms(100);
40         //Step 2
41         PORTB = 0b00001010; // RB3 and RB1 HIGH
42         delay_ms(100);
43         //Step 3
44         PORTB = 0b00000110; // RB2 and RB1 HIGH
45         delay_ms(100);
46         //Step 4
47         PORTB = 0b00000101; // RB2 and RB0 HIGH
48         delay_ms(100);
49     }
50 }
51 return;
52 }

```

Page 2

Figure 11

10. Rotary Encoder GUI Using Python (Tkinter + PySerial)

10.1 Program Objective

To develop a Python-based graphical user interface that receives rotary encoder count data from a microcontroller via UART serial communication and display it in real Time.

10.2 Import of Required Libraries

The following libraries are imported to enable serial communication, COM port detection, and GUI creation.

```
python

import serial
import serial.tools.list_ports
import tkinter as tk
```

10.3 COM Port Detection

The program scans and displays all available COM ports connected to the system to identify the correct serial interface.

```
python

ports = list(serial.tools.list_ports.comports())
print("Available COM ports:")
for p in ports:
    print(p.device)
```

10.4 Serial Port Configuration

After identifying the correct COM port, serial communication is initialized with predefined parameters. Exception handling is implemented to prevent execution failure.

```
python

PORT = "COM3"

try:
    ser = serial.Serial(PORT, 9600, timeout=1)
except serial.SerialException as e:
    print("ERROR: Cannot open serial port")
    exit()
```

10.5 GUI Window Initialization

A Tkinter window is created with a fixed size and appropriate title for rotary Encoder monitoring.

```
root = tk.Tk(root.title("Rotary Encoder GUI"))
root.geometry("300x150")
```

10.6 Encoder Data Display Element

A label widget is added to the GUI to display the encoder count. Initially, it indicates That the system is waiting for data.

```
python

label = tk.Label(root, text="Waiting for data...", font=("Arial", 18))
label.pack(pady=40)
```

10.7 Serial Data Acquisition

A function is defined to continuously monitor the serial buffer and read incoming data from the microcontroller.

```
def read_serial():
    if ser.in_waiting:
        line = ser.readline().decode(errors="ignore").strip()
```

10.8 Data Processing and Validation

The received serial data is validated against a predefined format. If valid, the encoder count value is extracted.

```
python

if line.startswith("COUNT:"):
    value = line.split(":")[1]
```

10.9 Real-Time GUI Update and Execution

The extracted encoder count is displayed on the GUI. A timed callback mechanism ensures non-blocking and continuous updates. The serial read function is then initiated, and the Tkinter main loop is started to keep the GUI active.

```
label.config(text=f"Count: {value}")  
root.after(50, read_serial)  
    read_serial()  
root.mainloop()
```

11.Conclusion

The project titled “Display of Incremental Rotary Encoder Output on GUI Using PIC 16-bit Microcontroller” has been successfully designed and implemented. The project demonstrates effective interfacing of an incremental rotary encoder with a PIC 16-bit microcontroller for accurate position and direction sensing using quadrature encoder signals.

The microcontroller efficiently processes encoder outputs through its internal peripherals, including the Quadrature Encoder Interface (QEI), and transmits the processed count data to a personal computer using UART serial communication. The transmitted data is displayed in real time on a graphical user interface (GUI) developed using Python, Tkinter, and PySerial, providing a clear and user-friendly visualization of encoder output.

In addition, the project includes the simulation and demonstration of unipolar and bipolar stepper motor control, highlighting the practical application of incremental encoder feedback in motion control systems. The integration of encoder theory, microcontroller operation, serial communication, motor control simulation, and GUI-based monitoring validates the robustness and flexibility of the proposed system.

Overall, the project meets all stated objectives and provides a reliable, low-cost, and scalable solution for monitoring incremental rotary encoder outputs. The implemented system forms a strong foundation for future developments such as closed-loop motor control, speed measurement, and advanced automation applications.

